

Adaptación de Autómatas Celulares en GPU para Simulaciones en Mecánica Estadística

Documentación del Repositorio Base

7 de marzo de 2026

Índice

1. Introducción y Preparación del Entorno	2
1.1. Depuración del Repositorio	2
1.2. Gestión del Entorno Virtual	2
2. Entorno Gráfico y Control de la Simulación	2
2.1. Modos de Ejecución	2
2.2. Monitorización del Progreso	3
3. El Modelo Base: Autómata Celular de Greenberg-Hastings	3
3.1. Estados Discretos del Sistema	3
3.2. Dinámica de Activación y Umbrales	4
4. Interacción del Usuario y Condiciones Iniciales	4
4.1. Control Interactivo en Tiempo Real	4
4.2. Importación de Patrones Computacionales (.PNG)	4
5. Arquitectura Computacional: Aceleración por GPU y <i>Shaders</i>	5
5.1. Paralelización Mediante Mapeo de Texturas	5
5.2. Codificación de Variables de Estado en Canales RGBA	5
5.3. Inyección de Parámetros y <i>Display Shader</i>	6
6. Propuesta de Adaptación: Dinámica de Incendios Forestales	6
6.1. Modificación de la Fase Refractaria Absoluta	6
6.2. Transición a un Modelo Pseudo-Continuo: Implementación del Viento . . .	6
6.3. Integración de Topología y Orografía	7

1. Introducción y Preparación del Entorno

El presente trabajo parte de la bifurcación (*fork*) de un repositorio previo desarrollado como Trabajo de Fin de Grado (TFG). Dicho código base contiene diversas implementaciones de modelos computacionales acelerados por hardware. Para adaptar este repositorio a los objetivos específicos de nuestro proyecto de Mecánica Estadística —centrado en la propagación de estados discretos y pseudo-continuos— es necesario realizar un proceso previo de depuración y configuración del entorno de trabajo.

1.1. Depuración del Repositorio

El código original incluye implementaciones ajenas al alcance de este estudio que deben ser descartadas para optimizar el flujo de trabajo. En concreto, se procederá a la eliminación de:

- **El Juego de la Vida (*Conway's Game of Life*):** Modelo de autómatas celulares estándar no requerido en esta simulación.
- **Modelos Neuronales Continuos:** Implementaciones referidas en el código como sistemas de ecuaciones diferenciales continuas (diseñados originalmente para simular propagación de señales en materia gris/blanca cerebral, tumores y fenómenos de resonancia estocástica).

Tras la limpieza, el archivo principal sobre el que pivotará la simulación y que deberá ser ejecutado es el *script* denominado **main GPU mode**. Este archivo gestiona la inicialización de la malla y la llamada a los *shaders* encargados del cálculo paralelo.

1.2. Gestión del Entorno Virtual

Para garantizar la reproducibilidad del experimento y el correcto manejo de dependencias (especialmente aquellas relacionadas con la visualización y el cómputo en la tarjeta gráfica), se establece el uso de gestores de entornos virtuales como **Conda** o **Miniconda**. Esto facilitará la instalación ordenada de las librerías requeridas sin generar conflictos a nivel de sistema operativo.

2. Entorno Gráfico y Control de la Simulación

El control interactivo y la renderización de la simulación se llevan a cabo a través de una Interfaz Gráfica de Usuario (GUI) programada mediante el *framework* **PySide**. Esta herramienta no solo permite observar la evolución del sistema matemático en tiempo real, sino que ofrece flexibilidad para alternar entre la observación fenomenológica y la obtención de datos puros de manera eficiente.

2.1. Modos de Ejecución

El sistema cuenta con dos enfoques de simulación, ajustables según la necesidad del experimento:

1. **Modo de Visualización en Tiempo Real:** Diseñado para la inspección visual de la dinámica del sistema. Permite limitar la tasa de refresco a valores manejables (por ejemplo, 24 o 60 *frames* por segundo).
2. **Modo de Rendimiento Máximo (*Headless*):** Cuando el objetivo es alcanzar un estado estacionario o evaluar el resultado final tras un número elevado de iteraciones, la actualización visual supone un cuello de botella. Para ello, se oculta la renderización de la malla (`hide`). Sin embargo, para evitar que el sistema operativo (Windows) detecte la aplicación como un proceso inactivo o congelado, se mantiene una llamada continua a la función `process.events`, garantizando la estabilidad del *script*.

2.2. Monitorización del Progreso

Dado que en simulaciones extensas en modo *headless* se carece de retroalimentación visual, se ha integrado la librería `tqdm`. Esta herramienta provee una barra de progreso en la consola de comandos, indicando el porcentaje completado, el número de iteraciones restantes y el tiempo de cómputo por paso.

Para minimizar el impacto de esta monitorización en el rendimiento general de la CPU, la actualización de la barra de progreso no se realiza en cada paso temporal, sino que se condiciona matemáticamente mediante el operador módulo. Se comprueba que el resto de la división iterativa sea cero (ej. `if step% 100 == 0`), actualizando la consola únicamente cada cien iteraciones.

3. El Modelo Base: Autómata Celular de Greenberg-Hastings

El núcleo computacional del proyecto se basa en una implementación del modelo de Greenberg-Hastings. Este modelo matemático se inscribe dentro de la teoría de los autómatas celulares excitables, proporcionando una discretización del espacio físico (una malla o *grid*) estructurada en celdas que interactúan con su vecindad inmediata.

3.1. Estados Discretos del Sistema

A diferencia de los modelos continuos, el estado de cada celda en el autómata de Greenberg-Hastings se define mediante tres fases discretas fundamentales, las cuales son representadas visualmente mediante un código de colores específico:

- **Estado Activo o Excitado (Blanco):** La celda se encuentra en su fase de máxima energía o excitación. En el contexto de nuestro estudio de dinámica de propagación, equivale al momento en el que el frente de onda (o de fuego, en simulaciones forestales) alcanza dicha posición.
- **Estado Inactivo o de Reposo (Negro):** La celda carece de actividad en el instante actual, pero es susceptible de ser excitada si recibe el estímulo suficiente de su vecindad.

- **Estado Refractario (Rojo):** Una vez que la celda abandona el estado activo, entra en un periodo de recuperación durante el cual no puede volver a ser excitada, independientemente de la actividad de sus vecinos. La representación gráfica de este estado varía a lo largo de un gradiente de intensidad del color rojo, denotando el tiempo restante para volver al reposo. En la configuración base del código, este periodo está parametrizado a 15 iteraciones temporales.

3.2. Dinámica de Activación y Umbrales

La transición del estado de reposo al estado excitado está regida por un parámetro de *umbral de activación*. Este parámetro dicta el número mínimo de celdas vecinas en estado activo requeridas para desencadenar la excitación de una celda inactiva.

La parametrización mínima viable para este umbral es de un vecino activo. Si el umbral se estableciera de manera teórica en cero, el modelo perdería su coherencia fenomenológica, provocando el colapso numérico del sistema de reglas de propagación local.

4. Interacción del Usuario y Condiciones Iniciales

Para el diseño de experimentos en Mecánica Estadística, la capacidad de establecer condiciones iniciales de contorno arbitrarias es fundamental. La plataforma desarrollada ofrece herramientas dinámicas tanto para la interacción en tiempo real como para la importación sistemática de patrones predefinidos.

4.1. Control Interactivo en Tiempo Real

El entorno gráfico dotado por PySide está programado para interpretar entradas a través de los periféricos, permitiendo la manipulación directa de la topología de la malla:

- **Activación manual (Clic izquierdo):** Permite forzar el cambio de estado de una celda específica (de reposo a excitada, o viceversa) originando focos de propagación arbitrarios en plena ejecución.
- **Navegación espacial (Clic derecho):** Habilita el desplazamiento (*pan*) a través del espacio de simulación, crucial para inspeccionar mallas de alta resolución.
- **Bloqueo topológico (Rueda central del ratón):** Asigna a la celda una condición de frontera estática (bloqueo o muro). Las celdas bloqueadas no interactúan con el entorno, forzando a las ondas de propagación a rodear el obstáculo impuesto, alterando la geometría del frente de excitación.

4.2. Importación de Patrones Computacionales (.PNG)

Además de la interacción manual, el sistema soporta la lectura y traducción de archivos de imagen en formato PNG para cargar mallas de inicio preconfiguradas. Cada píxel de la imagen importada se mapea a una celda del autómatas, lo que permite introducir geometrías complejas.

Como demostración de la completitud Turing de los autómatas celulares, el repositorio base incluye ejemplos de puertas lógicas (tales como la puerta NOT). Esta funcionalidad requiere que ciertas regiones del patrón se mantengan bajo una condición de energía o activación constante, anclando de forma estática su estado para sostener la lógica del circuito sobre la malla.

5. Arquitectura Computacional: Aceleración por GPU y *Shaders*

El principal desafío en la simulación de sistemas estadísticos basados en redes bidimensionales es el coste computacional asociado a la evaluación iterativa de vecindades. En las primeras iteraciones del proyecto base, el cálculo secuencial a través de la Unidad Central de Procesamiento (CPU) demostró ser un cuello de botella severo, colapsando el sistema al intentar resolver mallas de tamaño moderado (100x100 celdas).

Para solventar este límite, la arquitectura del software fue rediseñada para aprovechar el paralelismo masivo de la Unidad de Procesamiento Gráfico (GPU) mediante el uso de *shaders* programables.

5.1. Paralelización Mediante Mapeo de Texturas

El núcleo del cálculo matemático se ha desplazado a un *Fragment Shader*. En este paradigma, la malla completa de la simulación se trata como una textura bidimensional (objeto `sampler2D`), donde cada celda del autómata celular se corresponde unívocamente con un píxel de dicha textura.

Dado que la GPU cuenta con miles de hilos de ejecución (*threads*), el estado de cada celda se actualiza simultáneamente en un único ciclo de reloj. Esto permite escalar la resolución del modelo a mallas densas (por ejemplo, 500x500 celdas) manteniendo un rendimiento fluido y en tiempo real.

5.2. Codificación de Variables de Estado en Canales RGBA

Al procesarse como una textura, los parámetros físicos del sistema se deben abstraer y almacenar en los canales de color (Rojo, Verde, Azul y Alfa). Para evitar inestabilidades numéricas de coma flotante y optimizar el rendimiento del *shader*, todas las variables operan en el dominio continuo $[0,0,1,0]$ (*floats*), incluso aquellas de naturaleza booleana. El mapeo se define de la siguiente manera:

- **Canal Rojo (R - *Voltaje* / V_{pixel}):** Actúa como la variable de estado principal de la celda.
 - $V = 1,0$: Celda activa/excitada.
 - $0,0 < V < 1,0$: Celda en estado refractario. En cada paso temporal, su valor decrece linealmente según una constante de decaimiento ($V_{new} = V_{old} - decay$).
 - $V = 0,0$: Celda en reposo.

- **Canal Azul (B - Bloqueo Topológico):** Opera como un pseudo-booleano. Si el valor del canal es mayor a 0,5, el sistema interpreta que la celda es un obstáculo infranqueable (muro) e inhibe cualquier interacción con su vecindad.
- **Canal Alfa (A - Opacidad):** Se mantiene constante en un valor de 1,0 para asegurar la visibilidad de la textura.

5.3. Inyección de Parámetros y *Display Shader*

La parametrización macroscópica (tamaño de la cuadrícula, umbral de excitación) se transfiere desde el *script* principal en Python al *shader* a través de variables de tipo **uniform** (denotadas habitualmente con el prefijo `Ü`). El entorno maneja tipos de datos vectoriales nativos de GLSL (`vec2`, `vec3`, `vec4`).

Cabe destacar que los colores crudos utilizados para los cálculos internos (RG) no son los que se presentan visualmente al usuario. El código implementa un segundo *shader* secundario (*Display Shader*) que traduce el estado lógico de los canales en la paleta de colores previamente descrita (blanco, negro, tonos de rojo y gris).

6. Propuesta de Adaptación: Dinámica de Incendios Forestales

Para alinear el repositorio base con los objetivos del trabajo de Mecánica Estadística, se propone la adaptación del autómata celular excitado hacia un modelo epidemiológico o de propagación de incendios forestales, alterando tanto las fases refractarias como la discretización espacial.

6.1. Modificación de la Fase Refractaria Absoluta

En la simulación forestal, el estado refractario cíclico pierde sentido físico. Se procederá a alterar la lógica del *shader* computacional para que, una vez que la celda transite del estado activo (fuego) al estado inactivo, quede marcada permanentemente como "quemada" (ceniza).

A nivel algorítmico, esto se implementará forzando a la celda a adoptar un estado de bloqueo permanente (similar a la activación del canal Azul o definiendo un color específico de muerte) que la elimine del sumatorio de interacciones futuras de la red.

6.2. Transición a un Modelo Pseudo-Continuo: Implementación del Viento

El mayor aporte técnico radicarán en transformar el umbral de activación puramente discreto (conteo entero de vecinos activos) en una evaluación de un *potencial continuo*. Esto permitirá introducir dinámicas de anisotropía direccional espacial, como la acción del viento.

La ecuación de actualización del estado dependerá ahora de una suma ponderada de los potenciales de los vecinos. Para modelar el viento, se introducirá una matriz topológica

de pesos de 3×3 (o un vector direccional). Por ejemplo, si se establece un flujo de viento con dirección Norte, las constantes de proporcionalidad de transmisión hacia las celdas ubicadas en la componente Norte aumentarán, mientras que las de la componente Sur disminuirán. El estado de la celda central superará el umbral crítico si y solo si la suma de las contribuciones direccionales ponderadas de sus vecinos lo excede.

6.3. Integración de Topología y Orografía

Finalmente, aprovechando la infraestructura existente de importación de patrones PNG y el uso de canales libres (como el Verde), se prevé cargar mapas orográficos en tiempo de ejecución. De esta manera, las variaciones de intensidad en dichos canales representarán la pendiente o el tipo de vegetación, actuando como modificadores locales sobre la constante de propagación del fuego evaluada iterativamente por la GPU.